

Mesterséges intelligencia (GKLB_INTM124)

Ipari gépek meghibásodásának előrejelzése

Pados Kata
BSWC2A
mérnökinformatikus BSc, levelező

Tartalom

Feladat bemutatása	2
Az adathalmaz	2
Adatok előkészítése	3
Adatok felosztása:	3
Skálázás:	3
Neurális háló felépítése	3
A modell tanítása	4
Tanítási beállítások (model.compile())	4
Tanítási paraméterek (model.fit())	5
Hiperparaméterek vizsgálata	5
Interaktív előrejelzés	6
Összegzés	7
Források	8

Feladat bemutatása

A cél egy gépi tanulási feladat megoldása neurális hálók segítségével. A feladat pedig a következő: ipari gépek meghibásodásának előrejelzése szenzoradatok alapján.

Felhasznált könyvtárak

pandas: gyors, rugalmas és jól használható adatszerkezeteket biztosít a strukturált adatok egyszerű és hatékony kezeléséhez. Adatok beolvasására, tisztítására, módosítására és az adathalmazok kezelésére került felhasználásra. [1]

scikit-learn: gépi tanulási könyvtár, különböző adatelemzési és előfeldolgozási eszközöket biztosít. Az adatok felosztására, skálázására és a modell kiértékelésére került felhasználásra.[2]

tensorflow: gépi tanulási keretrendszer, megkönnyíti az ml modellek létrehozását és futtatását különböző környezetekben. A tensorflow részét képező Keras API került felhasználásra neurális háló létrehozásához, tanításához és kiértékeléséhez. [3], [4]

Az adathalmaz

A feladat során az AI4I 2020 Predictive Maintenance Datasetet [5] használtam fel, amely az iparban előforduló karbantartási adatokat utánozza. Az adathalmazban 10.000 szintetikus adatokból álló adatsor található. Minden sor 14 jellemzőt tartalmaz:

- UID: egyedi azonosító (1-10000)
- Product ID: L (low) vagy M (medium) vagy H (high) és szériaszám.
- Type: L, M vagy H (minőség)
- Air temperature: levegő hőmérséklete (K)
- Process temperature: a folyamat hőmérséklete (K)
- Rotational speed: forgási sebesség (rpm)
- Torque: nyomaték (Nm)
- Tool wear: szerszám kopása percekben
- Machine failure: történt-e meghibásodás (0 = nem, 1 = igen)
- TWF: tool wear failure (szerszámkopás miatti hiba)
- HDF: heat dissipation failure (hőelvezetési hiba)
- PWF: power failure (teljesítmény hiba)
- OSF: overstrain failure (túlterhelés miatti hiba)
- RNF: random failure (véletlen hiba)

Adatok előkészítése

A „fölösleges” dolgok eltünttetése:

1. Üres sorok eltávolítása
2. Duplikált sorok eltávolítása
3. Hibatípus adatok eltávolítása (nem kerültek felhasználásra)
4. Azonosító jellegű oszlopok eltávolítása

One hot encoding:

Type lehetséges értékei: L, M és H. Mivel numerikus adatokkal kell dolgoznunk, ezért három új bináris oszlop lett létrehozva: L, M és H, ahol az adott kategóriának megfelelő oszlop értéke 1, a többi pedig 0.

Adatok felosztása:

A bemeneti és célváltozók meghatározása után az adatok tanító és teszt halmazokra lettek osztva. A bemeneti változókhoz minden, kivéve a 'Machine failure', míg a célváltozóhoz csak a 'Machine failure' tartozik. Az adathalmaz felosztás 70-30%-os arányban történt, ahol az adatok 70%-a a modell tanítására, a 30%-a pedig a modell teljesítményének értékelésére lett használva.

Skálázás:

A skálázás előtt megvizsgáltam az egyes változók minimum- és maximum értékeit, amelyek az 1. táblázatban láthatók. Az adatokból megfigyelhető, hogy a különböző jellemzők értéktartománya jelentősen eltér egymástól, ezért a megfelelő modellműködés érdekében szükséges a skálázás alkalmazása.

	air temp	proc temp	rot speed	torque	tool wear
min	295.3	305.7	1168.0	3.8	0.0
max	304.5	313.8	2886.0	76.6	253.0

1. táblázat

Neurális háló felépítése

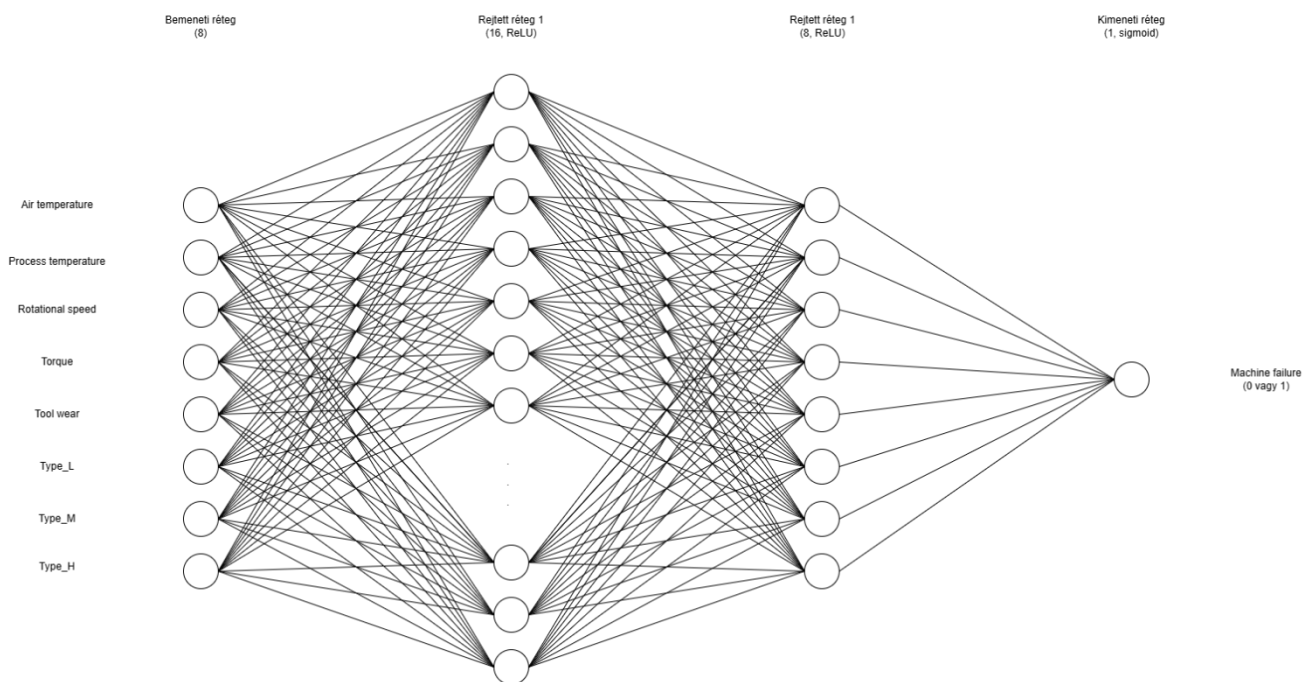
A feladat megoldásához többrétegű perceptront (MLP-MultiLayer Perceptron) alkalmaztam, amely az egyik leggyakrabban használt neurális hálózati architektúra. Előrecsatolt (feed-forward) működésű hálózat, a bemeneti adatoktól a kimenet felé halad, visszacsatolás nélkül. A bemeneti és kimeneti rétegek között rejtett réteg(ek) található. Ezek végzik a bemeneti adatok feldolgozását.

A több rejtett réteg segítségével a neurális hálózat képes bonyolultabb mintázatok felismerésére a szenzoradatokban. Ehhez nemlineáris aktivációs függvények használata is szükséges, különben a modell nem tudná megfelelően megtanulni az adatok közötti összetettebb kapcsolatokat. [6]

Az alkalmazott neurális háló felépítése az 1. ábrán látható. A modell 8 bemeneti jellemzőt fogad, amelyeket két rejtett réteg dolgoz fel. Az első rejtett réteg 16, a második pedig 8 neuront tartalmaz. A rejtett rétegek ReLU, míg a kimeneti réteg sigmoid aktivációs függvényt alkalmaz.

ReLU (Rectified Linear Unit) a negatív bemeneti értékek esetén 0-t, a pozitív értékek esetén pedig a bemeneti értéket adja tovább változatlanul. Gyors és hatékony tanítást tesz lehetővé. [7]

Sigmoid függvény alacsony érték (<-5) esetén 0-hoz közeli értéket ad, magas érték esetén (>5) 1-hez közeli ad. Így a kimeneti érték 0 és 1 közé esik, ami valószínűségként értelmezhető. [8]



1. ábra

A modell tanítása

Tanítási beállítások (`model.compile()`)

A neurális háló tanításához az Adam optimalizáló algoritmust, bináris keresztentropia veszteségfüggvényt és accuracy metrikát alkalmaztam.

Adam optimizer algoritmus egy sztochasztikus gradienstsökkentésen alapuló módszer, amely az első- és másodrendű momentumok adaptív becslését alkalmazza a modell tanítása során [9]

Binary crossentropy veszteségfüggvény a valós és a prediktált címkék közötti keresztentropia veszteséget számítja ki. [10]

Accuracy metrika azt mutatja meg, hogy a predikciók milyen gyakran egyeznek meg a valódi címkékkel. [11]

Tanítási paraméterek (model.fit())

A batch size paraméter meghatározza, hogy egy tanítási lépés során hány minta kerüljön feldolgozásra egyszerre.

Az epochs paraméter azt adja meg, hogy a modell hányszor halad végig a teljes tanító adathalmazon.

A validation split paraméter a tanító adatok egy részét validációs célra különíti el a tanítás közbeni kiértékeléshez.

A shuffle paraméter meghatározza, hogy az adatok minden epoch előtt összekeverésre kerüljenek-e.

A verbose paraméter a tanítás során megjelenő kimenet részletességét szabályozza.[12]

Hiperparaméterek vizsgálata

A modell teljesítményének vizsgálatához különböző tanítási paraméterek kerültek kipróbálásra. Az egyes konfigurációk eredményeit a 2. táblázat mutatja be. A különböző konfigurációk összehasonlítása során a modell teljesítményének értékelésére a loss és accuracy értékek kerültek felhasználásra. Az accuracy azt mutatja meg, hogy a modell predikciói milyen arányban egyeznek meg a valódi címkékkel, míg a loss a prediktált és a valós értékek közötti eltérést méri. A jobb modellre magasabb accuracy és alacsonyabb loss érték jellemző.

batch_size	epochs	validation_split	Loss	Accuracy	avg. loss	avg. accuracy
32	5	0.2	0,0882	97,20%	0,1024	96,56%
			0,1178	96,00%		
			0,1020	96,47%		
32	10	0.2	0,0920	97,07%	0,0947	97,01%
			0,0986	96,87%		
			0,0934	97,10%		
32	10	0.3	0,0931	96,67%	0,0862	97,03%
			0,0863	97,03%		
			0,0792	97,40%		
64	10	0.3	0,0932	97,07%	0,0983	97%
			0,1013	96,83%		
			0,1004	97,10%		
32	50	0.3	0,0851	97,57%	0,0744	97,67%
			0,0732	97,67%		
			0,0679	97,77%		
32	100	0.3	0,0549	98,23%	0,0613	97,93%
			0,0642	97,90%		
			0,0649	97,67%		
16	100	0.3	0,0748	97,73%	0,0700	97,84%

			0,0649	97,80%		
			0,0702	98,00%		
128	100	0.3	0,0763	97,40%	0,0725	97,56%
			0,0683	97,87%		
			0,0730	97,40%		
32	100	0.1	0,0638	97,33%	0,0598	98,08%
			0,0579	98,20%		
			0,0577	98,71%		

2. táblázat

Minden konfiguráció három alkalommal került lefuttatásra, majd az eredmények átlagolásra kerültek. Erre azért volt szükség, mert a neurális háló tanítása során kisebb eltérések előfordulhatnak a véletlenszerű inicializálás és az adatok keverése miatt.

Az eredmények alapján az epochs növelése javította a modell teljesítményét, mivel magasabb epochs mellett csökkent a loss és növekedett az accuracy. A túl nagy batch size viszont nem eredményezett további javulást, sőt bizonyos esetekben enyhe teljesítménycsökkenés figyelhető meg. A legjobb átlagos eredmény a 32-es batch size, 100 epochs és 0,1 validation split konfiguráció érte el, ahol az átlagos pontosság 98,08% volt.

Confusion matrix

A confusion matrix olyan táblázat, amely megmutatja, hogy a modell előrejelzései milyen arányban egyeznek meg a valós címkékkel. Bináris osztályozás esetén meghatározható belőle a true negative, false negative, true positive és false positive predikciók száma.[13] Ezt felhasználva kiírtam, hogy hány darab, illetve a hibák hány százaléka volt false negative illetve false positive.

Interaktív előrejelzés

Továbbfejlesztésként kialakításra került egy lehetőség a usernek saját szenzoradatok megadására is. A program futása után a felhasználó eldöntheti, hogy szeretne-e saját adatokat megadni előrejelzés készítéséhez.

A program bekéri a gép típusát és a különböző szenzor adatokat. Az adatok ezután ugyanazon az előfeldolgozási lépéseken mennek keresztül, mint a tanító adatok (mint pl.: one hot encoding, skálázás).

A neurális háló előrejelzést készít, majd megadja a meghibásodás becsült valószínűségét százalékos formában.

Összegzés

A feladat során egy neurális hálózaton alapuló gépi tanulási modell került elkészítésre ipari gépek meghibásodásának előrejelzésére. Az adatok előkészítése után egy többrétegű perceptron (MLP) modell került kialakításra TensorFlow/Keras környezetben. A modell tanításához Adam optimalizáló algoritmus, binary crossentropy veszteségfüggvény és accuracy metrika került alkalmazásra. A különböző tanítási paraméterek vizsgálata során több konfigurációt is kipróbáltam. Az eredmények alapján megfigyelhető volt, hogy az epochs számának növelése javította a modell teljesítményét, míg a túl nagy batch size nem eredményezett további javulást. A legjobb konfiguráció 98% feletti pontosságot ért el a teszt adathalmazon.

Az elkészített modell alkalmasnak bizonyult a szenzoradatok alapján történő géphiba-előrejelzésre, és jól szemlélteti a neurális hálózatok alkalmazhatóságát prediktív karbantartási feladatok esetén.

Források

- [1] „pandas - Python Data Analysis Library”. Elérés: 2026. május 15. [Online]. Elérhető: <https://pandas.pydata.org/>
- [2] „scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation”. Elérés: 2026. május 15. [Online]. Elérhető: <https://scikit-learn.org/stable/>
- [3] „TensorFlow”, TensorFlow. Elérés: 2026. május 15. [Online]. Elérhető: <https://www.tensorflow.org/>
- [4] „Keras: Deep Learning for humans”. Elérés: 2026. május 15. [Online]. Elérhető: <https://keras.io/>
- [5] Unknown, „AI4I 2020 Predictive Maintenance Dataset”. UCI Machine Learning Repository, 2020. doi: 10.24432/C5HS5C.
- [6] L. Perescu-Popescu és N. E. Mastorakis, „Multilayer perceptron and neural networks”, *Wseas Trans. Circuits Syst.*, júl. 2009, doi: 10.5555/1639537.1639542.
- [7] „tf.keras.activations.relu | TensorFlow v2.16.1”. Elérés: 2026. május 10. [Online]. Elérhető: https://www.tensorflow.org/api_docs/python/tf/keras/activations/relu
- [8] „tf.keras.activations.sigmoid | TensorFlow v2.16.1”. Elérés: 2026. május 10. [Online]. Elérhető: https://www.tensorflow.org/api_docs/python/tf/keras/activations/sigmoid
- [9] K. Team, „Keras documentation: Adam”. Elérés: 2026. május 10. [Online]. Elérhető: <https://keras.io/api/optimizers/adam/>
- [10] K. Team, „Keras documentation: Probabilistic losses”. Elérés: 2026. május 10. [Online]. Elérhető: https://keras.io/api/losses/probabilistic_losses/
- [11] K. Team, „Keras documentation: Accuracy metrics”. Elérés: 2026. május 10. [Online]. Elérhető: https://keras.io/api/metrics/accuracy_metrics/
- [12] „tf.keras.Model | TensorFlow v2.16.1”, TensorFlow. Elérés: 2026. május 10. [Online]. Elérhető: https://www.tensorflow.org/api_docs/python/tf/keras/Model
- [13] „confusion_matrix”, scikit-learn. Elérés: 2026. május 15. [Online]. Elérhető: https://scikit-learn/stable/modules/generated/sklearn.metrics.confusion_matrix.html